

מטלה 4 – רשתות תקשורת

חלק A – Ping

את המטלה נעשה באמצעות oracle virtualbox עם Ubuntu גרסה 22.04 כנדרש. ברירת המחדל של ה-VM היא להשתמש במתאם רשת NAT אבל פוגע לנו במטלה בעיקר בבדיקה של TTL, לכן אנחנו משתמשים בחיבור חוטי ומגדירים את ה-VM ל-bridge adapter.

קיים קובץ Makefile לצורך קומפילציה נוחה של כל חלקי המטלה. ה-Makefile מאפשר לקמפל כל תוכנה בנפרד (ping, traceroute, port_scanning) או את כולן יחד באמצעות פקודת make, ללא תלות בין הקבצים:

- make – מקמפל את כל התוכניות (ping, traceroute, port_scanning).
- make ping – מקמפל רק את התוכנית ping מתוך ping.c.
- make traceroute – מקמפל רק את התוכנית traceroute מתוך traceroute.c.
- make port_scanning – מקמפל רק את התוכנית port_scanning מתוך port_scanning.c.
- make clean – מוחק את קבצי ההרצה שנוצרו ומנקה את התיקייה.

נעזרנו בחומרים השונים של הקורס, בעיקר הקוד שלמדנו בתרגולים 5-8.

נתחיל בכתיבת תוכנית פשוטה ב-C המקבלת את הפרמטרים הנדרשים במטלה, בודקת את התקינות שלהם, במידה והם אינם תקינים דרוק שגיאה מתאימה למשתמש.

```

int main(int argc, char **argv) {
    // Step 1: Read command line args
    char *address = NULL; // Where to ping
    long count = -1; // How many pings (-1 = infinite)
    int flood = 0; // Flood mode on/off

    int opt;
    while ((opt = getopt(argc, argv, "a:c:f")) != -1) {
        switch (opt) {
            case 'a':
                address = optarg; // Save destination address
                break;

            case 'c': {
                // Convert count from string to number
                char *end = NULL;
                errno = 0;
                long value = strtol(optarg, &end, 10);

                // Make sure the number is valid and positive
                if (errno != 0 || end == optarg || *end != '\0' || value <= 0) {
                    fprintf(stderr, "Invalid value for -c: %s\n", optarg);
                    return 1;
                }

                count = value;
                break;
            }

            case 'f':
                flood = 1; // No delay between pings
                break;
        }
    }

    // The -a flag is needed
    if (address == NULL) {
        fprintf(stderr, "Error: -a <address> is required\n");
        return 1;
    }

    // Prepare address and socket:
    struct sockaddr_in dest;
    // Clean the structure
    memset(&dest, 0, sizeof(dest));
    dest.sin_family = AF_INET;
    // Convert IPv4 string ("8.8.8.8") into binary form
    if (inet_pton(AF_INET, address, &dest.sin_addr) != 1) {
        fprintf(stderr, "Invalid IPv4 address: %s\n", address);
        return 1;
    }
}

```

סידרנו את כל הכתובות הרלוונטיות לנו במבני struct כמו שצריך.

בעת, ניקח את הקוד של Raw Socket שראינו בתרגול, ונבין גם אנחנו סוקט כזה. בעת נפתח RAW SOCKET ונוודא שכמובן זה הצליח (נזכיר שיש צורך בהרשאות מנהל בזמן הריצה, כלומר SUDO):

```

// Step 2: Create a raw socket for ICMP
int sock = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);
if (sock < 0) {
    perror("socket");
    fprintf(stderr, "Socket didnt run! make sure you use sudo!!!\n");
    return 1;
}

// Print log:
printf("Pinging %s with 64 bytes of data:\n", address);

close(sock);
return 0;

```

עבשיו נצטרך להכין הודעת ICMP בהתאם למה שלמדנו בקורס, נעין בנספח במטלה:

ICMP Echo Request Header																												
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
Type (8 for IPv4)									Code (Always 0)									Checksum (2 Bytes)										
Identifier (2 Bytes)															Sequence Number (2 Bytes)													
Payload																												

לצורך חישוב הchecksum נשתמש במתודה שיש בקוד של התרגולים, ונבנה את ההודעה כמובן עם struct icmp_hdr:

```
// Step 3: build and send the ICMP msg:

// Allocate buffer for the ICMP packet (header + payload)
unsigned char packet[ICMP_PACKET_SIZE];
// Clear the memory
memset(packet, 0, sizeof(packet));

// Treat the beginning of the buffer as an ICMP header
struct icmp_hdr *icmp = (struct icmp_hdr *)packet;

// Set ICMP type to Echo Request (ping)
icmp->type = ICMP_ECHO;
icmp->code = 0;
icmp->un.echo.id = htons((unsigned short) getpid() & 0xFFFF); // identifier
icmp->un.echo.sequence = htons(1); // first sequence number

// Fill ICMP payload (56 bytes) with simple data
// This makes the total packet size 64 bytes
for (int i = 0; i < ICMP_PAYLOAD_SIZE; i++) {
    packet[sizeof(struct icmp_hdr) + i] = (unsigned char)('A' + (i % 26));
}

// Compute checksum over the entire ICMP packet (header + payload)
icmp->checksum = 0;
icmp->checksum = checksum(packet, sizeof(packet));

// Send the packet to the destination (no ports in ICMP, so sockaddr_in is enough)
ssize_t sent = sendto(sock, packet, sizeof(packet), 0,
                      (struct sockaddr *)&dest, sizeof(dest));
if (sent < 0) {
    // Check if sending failed
    perror("sendto");
    close(sock);
    return 1;
}

// Debug message to confirm packet was sent
printf("Sent %zd bytes of ICMP Echo Request\n", sent);

close(sock);
return 0;
```

אז בנינו בזיכרון חבילת ICMP Echo Request מלאה (payload + header), חישובנו checksum, ושליחנו אותה ליעד באמצעות ה-raw socket עם sendto.

בשלב זה נמדד זמן התחלה באמצעות CLOCK_MONOTONIC (של time.h), נשלח את ה-ICMP Echo Request ונמתין עד 10 מילישניות לתשובה בעזרת poll. במקרה של timeout התוכנית תסתיים ותיסגר בהתאם לדרישות המטלה.

אם מתקבלת תשובה, נקבל חבילת IP (חבילה כזאת היא כמובן מגיעה בביטים), נחלץ את אורך ה-IP header ואת ה-ICMP header, נוודא שמדובר ב-Echo Reply התואם לבקשה שנשלחה (id ו-sequence), במידה והחבילה אינה תואמת לבקשה אז אנו מתעלמים ממנה.

נמדוד את זמן ה-RTT במילישניות (כפי שמקובל), ונציג למשתמש את זמן ה-RTT, מספר ה-icmp_seq וערך TTL.

```
// Step 3: send and get respond (aka: echo)
// Start counting time for RTT
struct timespec t_start, t_end;
clock_gettime(CLOCK_MONOTONIC, &t_start);

// Send the packet to the destination (no ports in ICMP, so sockaddr_in is enough)
ssize_t sent = sendto(sock, packet, sizeof(packet), 0,
                     (struct sockaddr *)&dest, sizeof(dest));
if (sent < 0) {
    // Check if sending failed
    perror("sendto");
    close(sock);
    return 1;
}

// Wait up to 10 seconds for a reply using poll()
// Use poll for calling us when the socket is readable
struct pollfd pfd;
pfd.fd = sock;
pfd.events = POLLIN;

int pr = poll(&pfd, 1, TIMEOUT_MS);
if (pr == 0) {
    // timeout
    printf("Request timeout (no reply within 10 seconds)\n");
    close(sock);
    return 0; // program ends here in this step
}
if (pr < 0) {
    // internal error
    perror("poll");
    close(sock);
    return 1;
}

// Receive the raw IP packet (it contains IP header + ICMP message)
unsigned char rcvbuf[RECV_BUF_SIZE];
struct sockaddr_in from;
socklen_t fromlen = sizeof(from);
ssize_t n = recvfrom(sock, rcvbuf, sizeof(rcvbuf), 0,
                     (struct sockaddr *)&from, &fromlen);
if (n < 0) {
    perror("recvfrom");
    close(sock);
    return 1;
}
```

```
// Close the timer
clock_gettime(CLOCK_MONOTONIC, &t_end);
// Calculate the time in milliseconds
double rtt = diff_ms(t_start, t_end);

// Parse IP header to find TTL and ICMP header offset
struct iphdr *ip = (struct iphdr *)rcvbuf;
int ip_header_len = ip->ihl * 4;
int ttl = ip->ttl;

// check that the received data is valid
if (n < ip_header_len + (sizeof(struct icmphdr))) {
    fprintf(stderr, "Received packet too short\n");
    close(sock);
    return 1;
}

// Find and create the ICMP package
struct icmphdr *ricmp = (struct icmphdr *)(rcvbuf + ip_header_len);

// Validate that this is our Echo Reply
// Echo Reply type = ICMP_ECHOREPLY (0) AND same ID AND same sequence (using ntohs to convert for B-I to L-I)
if (ricmp->type == ICMP_ECHOREPLY &&
    ntohs(ricmp->un.echo.id) == my_id &&
    ntohs(ricmp->un.echo.sequence) == my_seq) {

    // Convert the ip from binary to IP str:
    char from_ip[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &from_ip, from_ip, sizeof(from_ip));

    // Print the msg: "64 bytes from X: icmp_seq=1 ttl=117 time=5.98ms"
    printf("%d bytes from %s: icmp_seq=%u ttl=%d time=%.3fms\n",
           sent, from_ip, my_seq, ttl, rtt);
} else {
    // Could be another ICMP message; for now just show a debug line
    printf("[debug] got ICMP type=%d code=%d (not our echo reply)\n",
           ricmp->type, ricmp->code);
}

close(sock);
return 0;
```

בעת נשנה את הקוד שלנו כך שנוכל לשלוח יותר מבקשה אחת כל פעם. אז במקום שליחה חד פעמית של Echo Request, נסיף לולאה ראשית ששולחת מספר בקשות ICMP ברצף. הלולאה פועלת באופן הבא:

- אם לא קיבלנו את הדגל c, הלולאה תפעל אינסוף פעמים - עד שהשתמש ייצא עם CTRL+C (נוסיף תמיכה לזה שסגירת התוכנית תהיה באופן יציב, ולא ישר נסגור את התוכנית מבלי שנסגור את הסוקט, נשתמש ב-signal כפי שמוסבר בנספח במטלה).
- אם קיבלנו את הדגל c עם מספר, אז הלולאה תפעל לפי מספר הפעמים שהשתמש הזין.

בכל איטרציה של הלולאה נשלחת בקשה אחת עם מספר רצף (icmp_seq) עולה.

כעת מכיוון שאנו נשלח מספר בקשות ICMP ולא רק בקשה בודדת אז נבין מתודה ייעודית לבניית החבילה, כל ריצה של הלולאה נקרא למתודה זו (עם אותו identifier, מספר רצף עדכני, וchecksum יחושב מחדש בהתאם כמובן):

```
// Build packet per-seq (checksum must be recomputed each time)
static void build_icmp_packet(unsigned char packet[ICMP_PACKET_SIZE],
                             unsigned short my_id,
                             unsigned short my_seq)
{
    memset(packet, 0, ICMP_PACKET_SIZE);

    struct icmphdr *icmp = (struct icmphdr *)packet;
    icmp->type = ICMP_ECHO;
    icmp->code = 0;

    icmp->un.echo.id = htons(my_id);
    icmp->un.echo.sequence = htons(my_seq);

    for (int i = 0; i < ICMP_PAYLOAD_SIZE; i++) {
        packet[sizeof(struct icmphdr) + i] = (unsigned char)('A' + (i % 26));
    }

    icmp->checksum = 0;
    icmp->checksum = checksum(packet, ICMP_PACKET_SIZE);
}
```

כמו כן נתייחס כבר עכשיו לדגל F שאנו מקבלים מהמשתמש:

- אם הדגל לא קיים אז תמיד נבצע השהייה של שנייה בין שליחת הבקשות עם sleep(1).

- אם הדגל קיים אז נשלח את החבילות ברצף ללא כל השהייה.

כאמור נשתמש ב-signal כך נוכל לקבל מהמשתמש את CTRL+C לצורך סגירת התוכנית כמו שצריך (לרבות סגירת הסוקט), נבין את מתודה העזר הבאה, עם הפלאג הבולייני הבא:

```
// Install SIGINT handler (Ctrl+C)
struct sigaction sa;
memset(&sa, 0, sizeof(sa));
sa.sa_handler = on_sigint;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
sigaction(SIGINT, &sa, NULL);

// Install SIGINT handler (Ctrl+C)
struct sigaction sa;
memset(&sa, 0, sizeof(sa));
sa.sa_handler = on_sigint;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
sigaction(SIGINT, &sa, NULL);

// Stop flag for Ctrl+C (SIGINT)
static volatile sig_atomic_t g_stop = 0;

// Signal handler (do NOT print here)
static void on_sigint(int signo) {
    (void)signo;
    g_stop = 1;
}
```

נגדיר signal handler עבור SIGINT, בעת לחיצה על CTRL+C הhandler מגדיר דגל עצירה (g_stop), ואז הלולאה הראשית בודקת את הדגל הזה ויוצאת מהלולאה ומהתוכנית בצורה מסודרת.

כמו כן נוסיף את המשתנים הבאים לצורך איסוף הסטטיסטיקה לסשן ping (כמה נשלח, כמה התקבל, RTT) וגם מדידת זמן התחלת הסשן (ע"י CLOCK_MONOTONIC) כדי לחשב את משך הריצה הכולל:

```

// Session statistics
long transmitted = 0;
long received = 0;

double rtt_min = 0.0;
double rtt_max = 0.0;
double rtt_sum = 0.0;

// Session timer for total time
struct timespec session_start;
clock_gettime(CLOCK_MONOTONIC, &session_start);

unsigned short my_id = (unsigned short)(getpid() & 0xFFFF);
unsigned short my_seq = 1;

// Reusable buffers
unsigned char packet[ICMP_PACKET_SIZE];
unsigned char recvbuf[RECV_BUF_SIZE];

```

כמו כן נבין משתנה לצורך שמירה על מספר המזהה של שליחת ה-PING שלנו: `my_id`. וגם משתנה לצורך מספר הבקשות שאנו שולחים: `my_seq`. כך נוכל לקשר בין תשובה המתקבלת לבין בקשה ששלחנו.

לצורך שליחה וקבלה מחזורית של כל הבקשות נצטרך ליצור `buffer` גם לשליחה וגם לקבלה. יצרנו שני `buffers` קבועים הניתנים לשימוש חוזר לצורך בניית חבילות ושליפת תשובות בכל ריצה של הלולאה, ללא הקצאת זיכרון חוזרת.

נבנה לולאה ראשית ונשים בה את השליחה והאימות שעשינו עד כה עם חבילה אחת, נשלב את השינויים והעדכונים עם הפרמטרים כפי שהסברנו קודם:

```

// Main loop - send multiple pings
while (!g_stop && (count == -1 || transmitted < count)) {

    // Build packet for this seq (checksum changes per seq)
    build_icmp_packet(packet, my_id, my_seq);

    // Start counting time for RTT
    struct timespec t_start, t_end;
    clock_gettime(CLOCK_MONOTONIC, &t_start);

    // Send the packet to the destination
    ssize_t sent = sendto(sock, packet, sizeof(packet), 0,
        (struct sockaddr *)&dest, sizeof(dest));
    if (sent < 0) {
        perror("sendto");
        break;
    }

    transmitted++;

    // Wait up to 10 seconds for a reply using poll()
    struct pollfd pfd;
    memset(&pfd, 0, sizeof(pfd));
    pfd.fd = sock;
    pfd.events = POLLIN;

    int pr = poll(&pfd, 1, TIMEOUT_MS);
    if (pr == 0) {
        printf("Request timeout for icmp_seq %u\n", my_seq);
        break; // NOT continue
    }
    if (pr < 0) {
        if (errno == EINTR) {
            // Interrupted by signal, loop will stop via g_stop
            continue;
        }
        // internal error
        perror("poll");
        break;
    }

    // Receive the raw IP packet (it contains IP header + ICMP message)
    struct sockaddr_in from;
    socklen_t fromlen = sizeof(from);
    ssize_t n = recvfrom(sock, recvbuf, sizeof(recvbuf), 0,
        (struct sockaddr *)&from, &fromlen);
    if (n < 0) {
        if (errno == EINTR) {
            continue;
        }
        perror("recvfrom");
        break;
    }

    // Close the timer
    clock_gettime(CLOCK_MONOTONIC, &t_end);
    // Calculate the RTT in milliseconds.
    double rtt = diff_ms(t_start, t_end);

    // Parse IP header to find TTL and ICMP header offset
    struct iphdr *ip = (struct iphdr *)recvbuf;
    int ip_header_len = ip->ihl * 4;
    int ttl = ip->ttl;

```

```

// check that the received data is valid
if (n < ip_header_len + (ssize_t)sizeof(struct icmphdr)) {
    fprintf(stderr, "Received packet too short\n");
    // Treat as "not received", continue
    my_seq++;
    if (!flood && !g_stop) sleep(1);
    continue;
}

// Find and create the ICMP package
struct icmphdr *ricmp = (struct icmphdr *) (recvbuf + ip_header_len);

// Validate that this is our Echo Reply
// Echo Reply type = ICMP_ECHOREPLY (0) AND same ID AND same sequence (using ntohs to convert for B-E to L-E)
if (ricmp->type == ICMP_ECHOREPLY &&
    ntohs(ricmp->un.echo.id) == my_id &&
    ntohs(ricmp->un.echo.sequence) == my_seq) {

    received++;

    // Update RTT stats
    if (received == 1) {
        rtt_min = rtt_max = rtt;
    } else {
        if (rtt < rtt_min) rtt_min = rtt;
        if (rtt > rtt_max) rtt_max = rtt;
    }
    rtt_sum += rtt;

    // Convert the ip form binary to IP str:
    char from_ip[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &from.sin_addr, from_ip, sizeof(from_ip));

    // Print the <msg>: "64 bytes from X: icmp_seq=1 ttl=117 time=5.98ms"
    ssize_t icmp_bytes = n - ip_header_len;

    printf("%zd bytes from %s: icmp_seq=%u ttl=%d time=%.3fms\n", icmp_bytes, from_ip, my_seq, ttl, rtt);
} else {
    // Could be another ICMP message; for now just show a debug line
    printf("[debug] got ICMP type=%d code=%d (not our echo reply)\n",
        ntohs(ricmp->type), ntohs(ricmp->code));
}

my_seq++;

// Flood mode means no sleep
if (!flood && !g_stop) {
    sleep(1);
}
}

```

הוספנו הלולאה שולחת מספר בקשות ICMP בהתאם לדגלים C ו-F, כאשר בכל איטרציה נבנית מחדש חבילת Echo Request עם מספר רצף חדש ו-checksum מעודכן. לאחר מכן, נמדד זמן בשביל ה-RTT, ובבצע `poll` בהמתנה לתשובה.

כמו כן במקום לתת למערכת ההפעלה ליצור בעצמה את ip header, הגדרנו בעצמו חבילה כזאת, כך שנוכל לשים TTL משלנו (64) ולא את ברירת המחדל של 255 (כפי שנאמר בהרצאה).

במקרה של timeout, התוכנית מדפיסה הודעה וסוגרת את לולאת שליחת ההודעות, ונדפיס את הסטטיסטיקות לבסוף.

אימות של תקינות החבילה שהתקבלה, חיפוש ה-TTL מתוך כותרת ה-IP, מציאת כותרת ה-ICMP, סינון תשובות לפי `id`, `type` ו-`sequence`, ועדכון סטטיסטיקות RTT ו-packet loss לאורך כל הריצה. גם כאן במידה והחבילה אינה תואמת לבקשה אז אנו מתעלמים ממנה.

הקוד כולל כעת טיפול תקין בסגירת התוכנית באמצעות CTRL+C כך שהריצה נעצרת בצורה מבוקרת.

כאמור במהלך ריצת התוכנית נאספים הנתונים הבאים:

- מספר החבילות שנשלחו (transmitted).
- מספר החבילות שהתקבלו (received).
- זמני RTT לכל Echo Reply.
- זמן ריצה כולל של הסשן.

בסיום הריצה (או לאחר CTRL+C) מודפסות הסטטיסטיקות הבאות:

- מספר חבילות שנשלחו והתקבלו.
- אחוז packet loss (חישוב פשוט שנוח לראות בעין בסיום השידור).
- זמן כולל.
- RTT מינימלי, ממוצע ומקסימלי

לבסוף נדפיס את הנתונים בצורה פשוטה:

```
// Print final statistics
struct timespec session_end;
clock_gettime(CLOCK_MONOTONIC, &session_end);
double total_ms = diff_ms(session_start, session_end);

printf("\n--- %s ping statistics ---\n", address);
printf("%ld packets transmitted, %ld received, ", transmitted, received);

long lost = transmitted - received;
double loss_pct = 0.0;
if (transmitted > 0) {
    loss_pct = (double)lost * 100.0 / (double)transmitted;
}
printf("%.1f%% packet loss, time %.0fms\n", loss_pct, total_ms);

if (received > 0) {
    double rtt_avg = rtt_sum / (double)received;
    printf("rtt min/avg/max = %.3f/%.3f/%.3f ms\n", rtt_min, rtt_avg, rtt_max);
}

close(sock);
return 0;
```

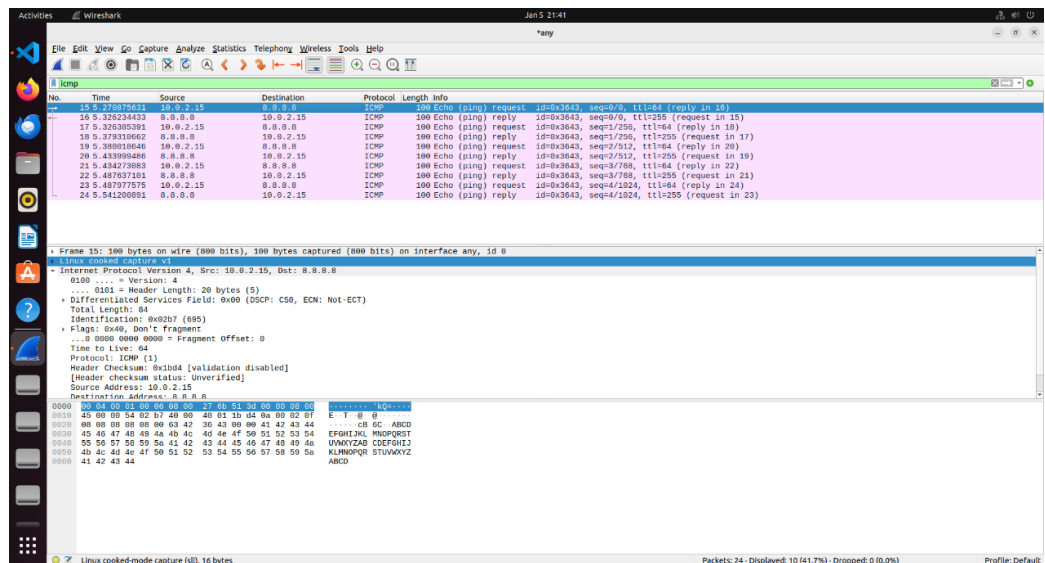
בעת נאזין לתעבורת הרשת, ונריץ את המקרים הבאים בהתאם לדרישות במטלה:

```
vboxuser@Ubuntu-22:~/Desktop/networks_4$ sudo ./ping -a 8.8.8.8 -c 5 -f
[sudo] password for vboxuser:
Pinging 8.8.8.8 with 64 bytes of data:
64 bytes from 8.8.8.8: icmp_seq=0 ttl=255 time=55.502ms
64 bytes from 8.8.8.8: icmp_seq=1 ttl=255 time=53.234ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=255 time=54.501ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=255 time=53.491ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=255 time=53.362ms

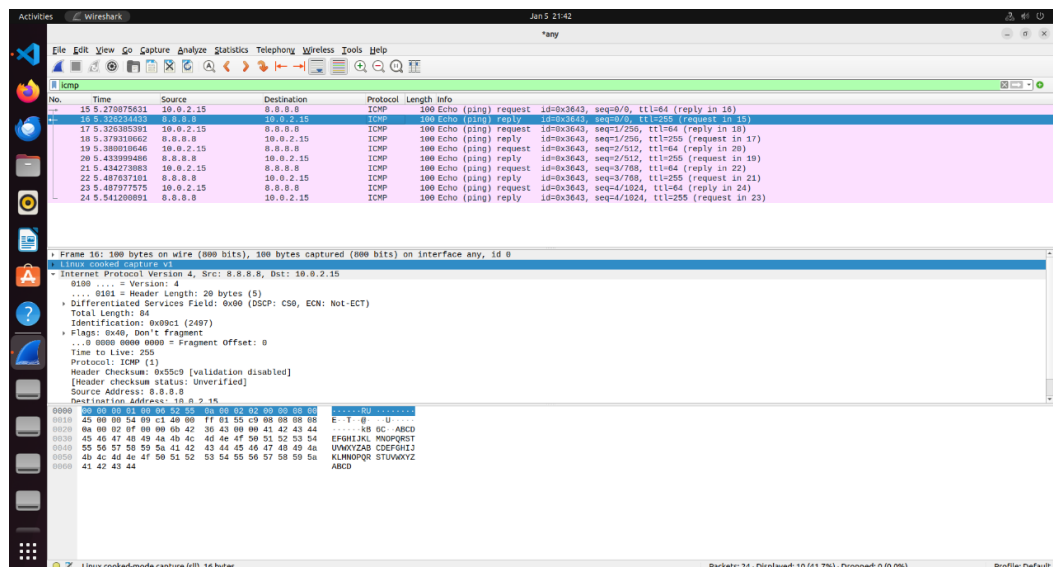
--- 8.8.8.8 ping statistics ---
5 packets transmitted, 5 received, 0.0% packet loss, time 271ms
rtt min/avg/max = 53.234/54.018/55.502 ms
```

נשים לב שאכן קיבלנו פלט תקין וההודעות שלנו נשלחו בנדרש. כעת נסניף את תעבורת הרשת באמצעות wireshark:

ההודעה שאנו שולחים:



התשובה שאנחנו מקבלים:

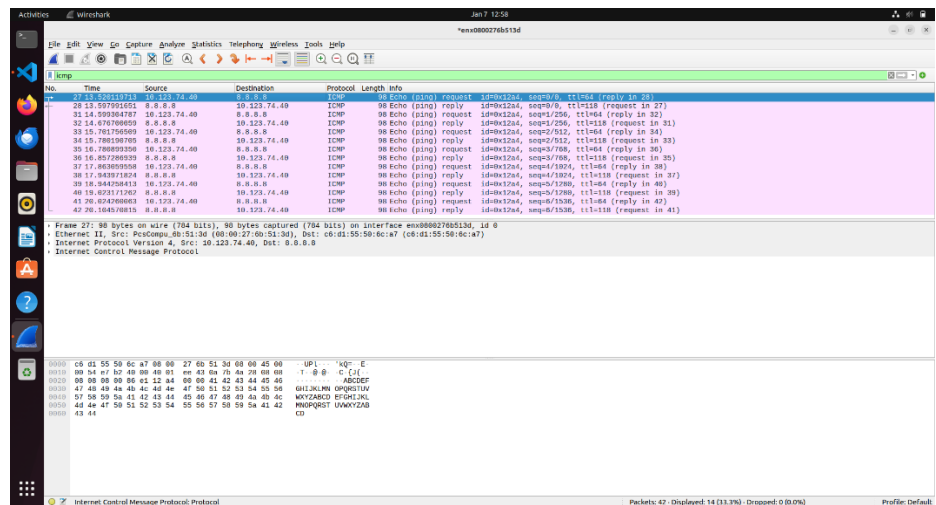


נוודא גם את שאר המקרים כמובן:

- בלי פלאגים נוספים רק כתובת IP עצמנו את התוכנית עם C+CTRL:

```
• vboxuser@Ubuntu-22:~/Desktop/networks_4$ sudo ./ping -a 8.8.8.8
[sudo] password for vboxuser:
Pinging 8.8.8.8 with 64 bytes of data:
64 bytes from 8.8.8.8: icmp_seq=0 ttl=118 time=78.095ms
64 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=77.551ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=118 time=78.620ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=118 time=76.652ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=118 time=81.105ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=118 time=79.321ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=118 time=80.560ms
^C
--- 8.8.8.8 ping statistics ---
7 packets transmitted, 7 received, 0.0% packet loss, time 7444ms
rtt min/avg/max = 76.652/78.843/81.105 ms
```

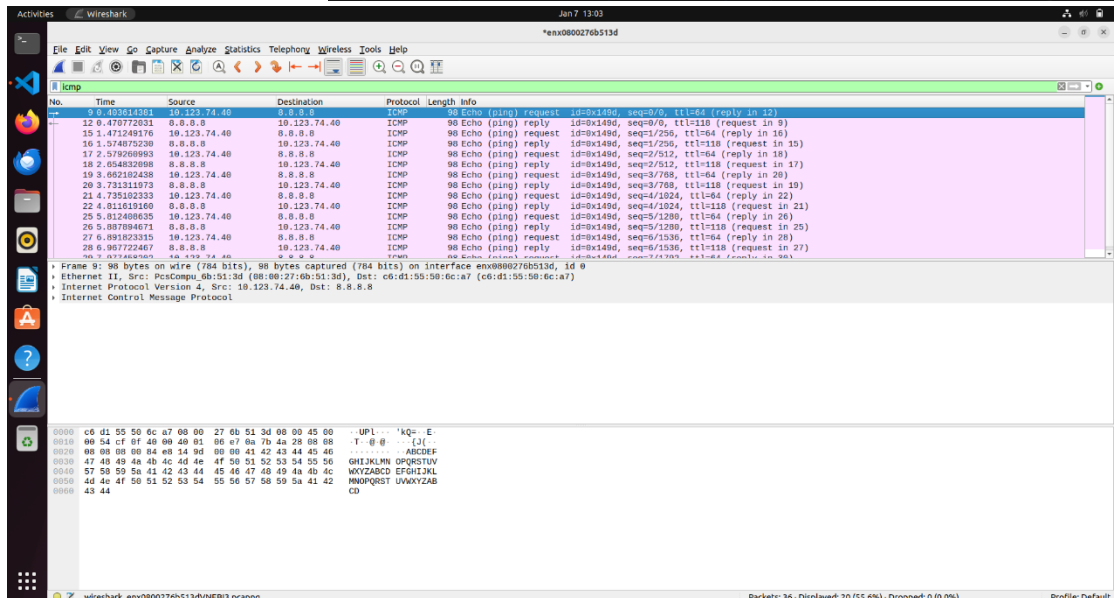
ניתן לשים לב לכל הפקטות שיוצאות ונכנסות בנדרש:



- בעת נוסף פרמטר C עם 10 כלומר לשלוח רק 10 הודעות:

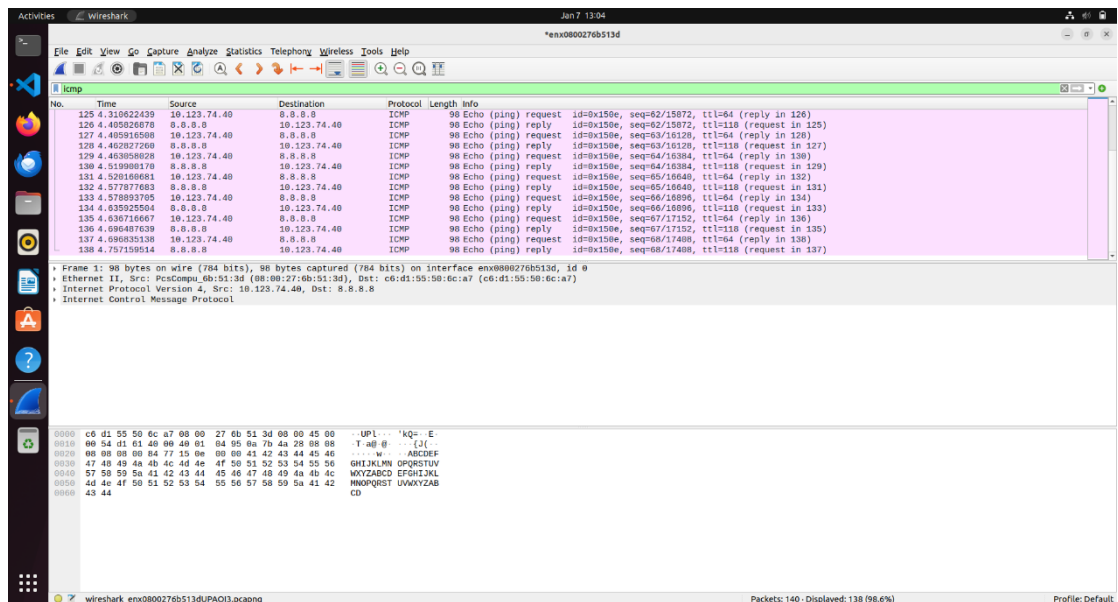
```
vboxuser@Ubuntu-22:~/Desktop/networks_4$ sudo ./ping -a 8.8.8.8 -c 10
Ping 8.8.8.8 with 64 bytes of data:
64 bytes from 8.8.8.8: icmp_seq=0 ttl=118 time=67.265ms
64 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=103.751ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=118 time=75.832ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=118 time=69.591ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=118 time=76.831ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=118 time=75.646ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=118 time=75.985ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=118 time=86.660ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=118 time=73.840ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=118 time=77.713ms

--- 8.8.8.8 ping statistics ---
10 packets transmitted, 10 received, 0.0% packet loss, time 10837ms
rtt min/avg/max = 67.265/78.311/103.751 ms
```



- וכעת פלאג F ששולח את ההודעות במקביל:

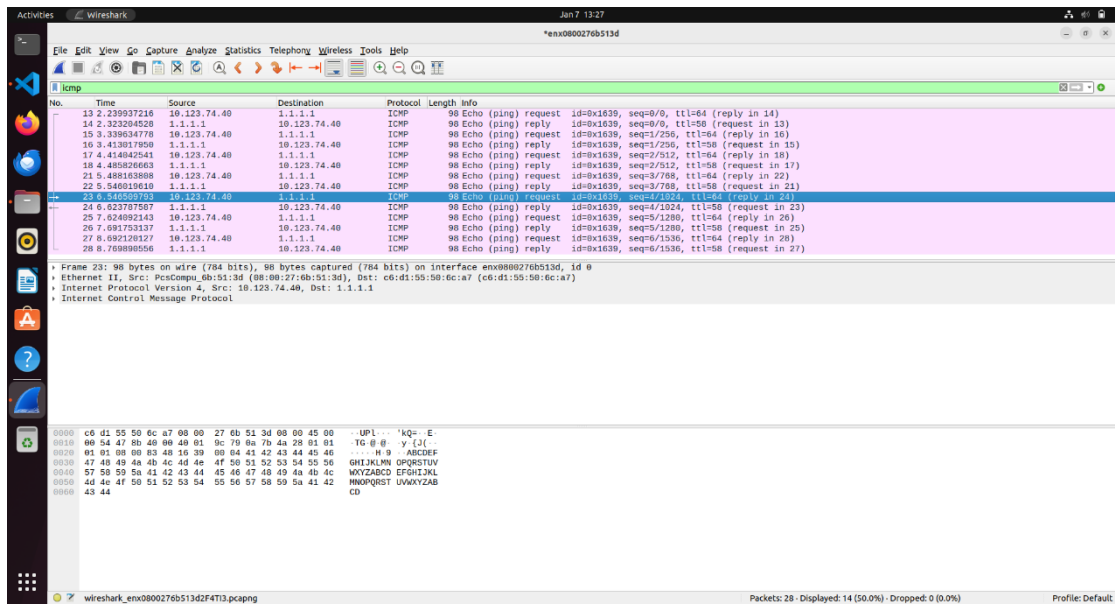
```
vboxuser@Ubuntu-22:~/Desktop/networks_4$ sudo ./ping -a 8.8.8.8 -f
64 bytes from 8.8.8.8: icmp_seq=52 ttl=118 time=61.323ms
64 bytes from 8.8.8.8: icmp_seq=53 ttl=118 time=58.028ms
64 bytes from 8.8.8.8: icmp_seq=54 ttl=118 time=98.160ms
64 bytes from 8.8.8.8: icmp_seq=55 ttl=118 time=57.874ms
64 bytes from 8.8.8.8: icmp_seq=56 ttl=118 time=59.872ms
64 bytes from 8.8.8.8: icmp_seq=57 ttl=118 time=57.457ms
64 bytes from 8.8.8.8: icmp_seq=58 ttl=118 time=88.243ms
64 bytes from 8.8.8.8: icmp_seq=59 ttl=118 time=57.312ms
64 bytes from 8.8.8.8: icmp_seq=60 ttl=118 time=61.368ms
64 bytes from 8.8.8.8: icmp_seq=61 ttl=118 time=58.489ms
64 bytes from 8.8.8.8: icmp_seq=62 ttl=118 time=95.270ms
64 bytes from 8.8.8.8: icmp_seq=63 ttl=118 time=57.117ms
64 bytes from 8.8.8.8: icmp_seq=64 ttl=118 time=57.079ms
64 bytes from 8.8.8.8: icmp_seq=65 ttl=118 time=58.712ms
64 bytes from 8.8.8.8: icmp_seq=66 ttl=118 time=57.367ms
64 bytes from 8.8.8.8: icmp_seq=67 ttl=118 time=60.040ms
^C
--- 8.8.8.8 ping statistics ---
69 packets transmitted, 68 received, 1.4% packet loss, time 4745ms
rtt min/avg/max = 57.079/68.865/112.691 ms
```



- ולשם הגיוון נעשה גם בדיקה עם שרת אחר, הפעם נבחר כתובת DNS אחרת של Cloudflare:

```
vboxuser@Ubuntu-22:~/Desktop/networks_4$ sudo ./ping -a 1.1.1.1
[sudo] password for vboxuser:
Ping 1.1.1.1 with 64 bytes of data:
64 bytes from 1.1.1.1: icmp_seq=0 ttl=58 time=83.542ms
64 bytes from 1.1.1.1: icmp_seq=1 ttl=58 time=73.664ms
64 bytes from 1.1.1.1: icmp_seq=2 ttl=58 time=72.062ms
64 bytes from 1.1.1.1: icmp_seq=3 ttl=58 time=58.095ms
64 bytes from 1.1.1.1: icmp_seq=4 ttl=58 time=77.439ms
64 bytes from 1.1.1.1: icmp_seq=5 ttl=58 time=67.754ms
64 bytes from 1.1.1.1: icmp_seq=6 ttl=58 time=77.854ms
^C
--- 1.1.1.1 ping statistics ---
7 packets transmitted, 7 received, 0.0% packet loss, time 6580ms
rtt min/avg/max = 58.095/72.916/83.542 ms
```

גם כאן ניתן להסניף את החבילות ולראות שהכל עובר כנדרש:



כמו כן נוודא שבדיקת הפרמטרים שלנו עובדת כנדרש ומחזירה שגיאה - כאן כתובת ה-IP אינה תקינה:

```
vboxuser@Ubuntu-22:~/Desktop/networks_4$ sudo ./ping -a 8.8.8.888 -c 5 -f
Invalid IPv4 address: 8.8.8.888
```

ובעת כאשר פרמטר a חסר:

```
vboxuser@Ubuntu-22:~/Desktop/networks_4$ sudo ./ping 8.8.8.888 -c 5 -f
Error: -a <address> is required
```

חלק B - Trace

בבנה כעת תוכנית אשר תדפיס את מסלול חבילת IP מהמחשב שלנו ליעד כלשהו, באמצעות חבילות ICMP Echo Request עם ערכי TTL עולים. כל פעם שנגיע לHUB כלשהו אז חבילת Echo תחזור אלינו.

גם כאן נשתמש באובסרווליר וירטואלי עם Ubuntu גרסה 22.04 עם אינטרנט קווי (bridge adapter) ולא NAT) כך החבילות יוצאות ישירות לרשת ובבדיקות באמת כל ה-hub שמופיע לנו במסלול המנותב של החבילה, ו-TTL יהיה אמיתי. גם כאן נעזר בשתי מתודות עזר שהיו גם בחלק הקודם לצורך חישוב checksum מתוך התרגולים, וגם המתודה diff_ms שתחשב לנו את הזמן של ה-RTT.

נתחיל בקליטת הפרמטרים מהמשתמש: אנו מקבלים פרמטר אחד שהוא כתובת היעד שלנו. בדומה לשלב A גם כאן יהיה פלאג a ולאחר מכן כתובת ה-IP של היעד, כפי שנדרש במטלה:

```
<sudo ./traceroute -a <destination_ip
```

כמובן גם כאן יש צורך ב-SUDO בשביל להשתמש ב-raw socket. במקרה שלא קיבלנו את פרמטר היעד תקין אז נדרוק שגיאה כנדרש ונסגור את התוכנית (כי אין מה לעשות בלי יעד בחיים כן?):

```

int main(int argc, char **argv) {
    // destination IP str:
    const char *dest_ip_str = NULL;

    // Get the parameters via the OPT functions:
    int opt;
    while ((opt = getopt(argc, argv, "a:")) != -1) {
        switch (opt) {
            case 'a':
                dest_ip_str = optarg;
                break;
            default:
                // we need 'a', else is not good:
                fprintf(stderr, "no 'a' parameter\n");
                return 1;
        }
    }

    if (!dest_ip_str) {
        // we need 'a'...
        fprintf(stderr, "no 'a' parameter\n");
        return 1;
    }
}

```

כעת נמיר את כתובת היעד שקיבלנו ממחרוזת למבנה IPv4 כפי שנדרש לשליחה: נאפס את struct, נגדיר שזה חיבור עם כתובת IPv4, וכמובן נמיר את הכתובת מטקסט לבינארי. אם ה-IP לא תקין אז נעצור עם הודעת שגיאה.

כמו כן, ניצור raw socket כפי שאנו כבר יודעים. גם כאן אנחנו נגדיר את ה-ip header בעצמנו (גם בשביל לא לתת למערכת ההפעלה להגדיר בעצמה TTL, כי אנחנו רוצים TTL של 64), בהמשך נבין את הערכים הרלוונטיים שנכנסים ל-ip header:

```

// Build destination address (standart IPv4)
struct sockaddr_in dest;
memset(&dest, 0, sizeof(dest));
dest.sin_family = AF_INET;

if (inet_pton(AF_INET, dest_ip_str, &dest.sin_addr) != 1) {
    // convert IP to str failed so IP not valid
    fprintf(stderr, "Invalid IPv4 address: %s\n", dest_ip_str);
    return 1;
}

// Create raw socket for sending custom IP packets
int sock = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);
if (sock < 0) {
    // Socket failed
    perror("socket(AF_INET, SOCK_RAW, IPPROTO_ICMP)");
    return 1;
}

// Tell kernel that we include the IP header ourselves
int on = 1;
if (setsockopt(sock, IPPROTO_IP, IP_HDRINCL, &on, sizeof(on)) < 0) {
    perror("setsockopt(IP_HDRINCL)");
    close(sock);
    return 1;
}

```

כעת נבין חבילת ICMP Echo Request בזיכרון, בדומה לחלק A:

- נבין struct של icmp_hdr.
- הגדרת type = ICMP_ECHO.
- code = 0.
- שימוש במזהה קבוע (my_id) המבוסס על getpid.
- מספר רצף (sequence) שעולה עם כל שליחה.
- הוספת payload קבוע (סתם שיהיה משהו בהודעה).
- חישוב checksum על כותרת ה-ICMP וה-payload.

כעת, לצורך זיהוי תשובות השייכות לבקשות שלנו:

- ניוצר מזהה my_id, המבוסס על getpid() ומוגבל ל-16 ביט באמצעות MASK
- נשמר מונה seq, אשר עולה בכל שליחה של חבילת ICMP

בנוסף, מודפסת למשתמש שורת פתיחה בסגנון traceroute, הכוללת:

- כתובת היעד
- מספר ה-hops המקסימלי
- מספר הניסיונות לכל hop

```
// id helps to tie a response to our request (using the "PID with AND 0xFFFF" for ICMP in 16-bit)
unsigned short my_id = (unsigned short)(getpid() & 0xFFFF);
// seq is raised on each probe, to identify each attempt
unsigned short seq = 1;

// Print log to user:
printf("traceroute to %s, %d hops max, %d probes per hop\n",
       dest_ip_str, MAX_HOPS, PROBES_PER_HOP);
```

בעת נשלח את כל הבקשות שלנו עם מעקב על מספר ה-TTL: לולאה הרצה על ערכי TTL מ-1 ועד MAX_HOPS (מוגדר בתור 30). עבור כל ערך TTL:

- נדפיס מספר hop.
- נשלח שלוש חבילות ICMP Echo Request (כפי שנדרש במטלה).
- נשמור דגל המציין האם הגענו ליעד הסופי.

כמו כן, נשלח מספר probes כפי שהכרנו בתרגול ובמטלה לכל hop כך קל לזהות איבוד חבילות וכך נשמור על מסלול יציב כנדרש.

עבור כל probe נבנית חבילה בזיכרון, הכוללת:

- בניית כותרת IP באמצעות struct iphdr:
 - גרסה: IPv4
 - אורך כותרת
 - אורך כולל של החבילה (IP + ICMP)
 - מזהה משתנה
 - TTL: הערך הנוכחי בלולאה
 - פרוטוקול: ICMP
 - כתובת יעד
 - חישוב checksum של כותרת ה-IP
- בניית הודעת ICMP Echo Request באמצעות struct icmp_hdr:
 - type = ICMP_ECHO
 - code = 0
 - identifier = my_id
 - sequence = seq (עולה בכל שליחה)
 - payload קבוע בגודל 56 בתים
 - חישוב checksum על כותרת ה-ICMP וה-payload

לאחר בניית הכותרות, מתקבל buffer אחד בזיכרון במבנה הבא:

[IP header][ICMP header][payload]

```
// Main loop over TTL/hops:
for (int ttl = 1; ttl <= MAX_HOPS; ttl++) {
    // For each hop print the ***
    printf("%2d ", ttl);
    fflush(stdout); // flush to refresh the screen each hop

    char hop_ip_first[INET_ADDRSTRLEN] = {0}; // Save the first IP returned with the same TTL, to print it once
    int reached_destination = 0; // Stop flag if we reached the destination

    // Each probe is an independent sending of an ICMP Echo Request packet with the same TTL,
    // Sending three probes improves reliability and allows for packet loss detection.
    for (int probe = 0; probe < PROBES_PER_HOP; probe++) {
        // Set a buffer for the packet we send
        unsigned char packet[FULL_PACKET_SIZE];
        memset(packet, 0, sizeof(packet)); // set all the val in the array to 0

        // Build IP header
        struct iphdr *ip = (struct iphdr *)packet;
        ip->version = 4; // IPv4...
        ip->ihl = (unsigned char)(IP_HEADER_SIZE / 4); // number of 32-bit words
        ip->tos = 0;
        ip->tot_len = htons((unsigned short)FULL_PACKET_SIZE);
        ip->id = htons((unsigned short)(my_id + ttl)); // any changing ID is fine
        ip->frag_off = htons(0);
        ip->tll = (unsigned char)ttl; // the TTL we send...
        ip->protocol = IPPROTO_ICMP;
        ip->saddr = 0;
        ip->daddr = dest.sin_addr.s_addr;

        ip->check = 0;
        ip->check = checksum(ip, IP_HEADER_SIZE);

        // Build ICMP Echo Request
        struct icmphdr *icmp = (struct icmphdr *)(packet + IP_HEADER_SIZE);
        icmp->type = ICMP_ECHO; // 8
        icmp->code = 0;
        icmp->un.echo.id = htons(my_id);
        icmp->un.echo.sequence = htons(seq++);

        // Payload: can be anything; we fill with a pattern...
        unsigned char *payload = (unsigned char *) (packet + IP_HEADER_SIZE + sizeof(struct icmphdr));
        for (int i = 0; i < ICMP_PAYLOAD_SIZE; i++) {
            payload[i] = (unsigned char)('A' + (i % 26));
        }

        icmp->checksum = 0;
        icmp->checksum = checksum(icmp, ICMP_PACKET_SIZE);
    }
}
```

לצורך חישוב ה-RTT, לפני שליחת החבילה נמדד זמן התחלה באמצעות `clock_gettime(CLOCK_MONOTONIC)`.

וכעת נשלח את החבילה שבנינו ב `sendto` בעזרת `buffer`. לאחר מכן נבצע המתנה לתשובה באמצעות `poll` עם `timeout` של שנייה אחת.

אם לא מתקבלת תשובה בזמן:

- מודפס "*"
- עוברים ל-`probe` הבא

```
// Send probe and measure time
struct timespec t_start, t_end;
clock_gettime(CLOCK_MONOTONIC, &t_start);

ssize_t sent = sendto(sock, packet, FULL_PACKET_SIZE, 0,
    (struct sockaddr *)&dest, sizeof(dest));
if (sent < 0) {
    // if the send failed we print '*' and try again
    perror("sendto");
    printf("* ");
    continue;
}

// Wait for a reply using poll() with a timeout (non-blocking wait)
struct pollfd pfd;
pfd.fd = sock; // The raw ICMP socket we are listening on
pfd.events = POLLIN; // We are interested only in incoming data

int pr = poll(&pfd, 1, TIMEOUT_MS);
if (pr <= 0) {
    // timeout or error: print '*' and try again
    printf("* ");
    fflush(stdout);
    continue;
}
```


כעת במידה וקיבלנו חבילה בתוך הזמן האפשרי, נאחסן את החבילה, נחלץ את הנתונים ונבדוק האם החבילה תקינה:

- קודם נמדד זמן RTT.
- נחלץ את ה-ip header.
- נחלץ את ה-ICMP header.

אם קיבלנו ICMP Echo Reply אז הגענו ליעד הסופי. נבדק שה-identifier תואם ל-my_id, ובמקרה כזה מסומן שהגענו ליעד – המסלול הושלם:

```
// stop the timer and calculate the RTT:
clock_gettime(CLOCK_MONOTONIC, &t_end);
double rtt = diff_ms(&t_start, &t_end);

// Parse outer IP header from received packet:
// Make sure we received at least a full IP header
if (n < (ssize_t)sizeof(struct iphdr)) {
    // not valid IP header so let's try again
    printf("* ");
    fflush(stdout);
    continue;
}

// Parse the outer IP header from the received packet
struct iphdr *rip = (struct iphdr *)recvbuf;
// IP header length is stored in 32-bit words, so multiply by 4
int rip_hlen = rip->ihl * 4;

// Verify that the packet also contains a full ICMP header
if (n < rip_hlen + (ssize_t)sizeof(struct icmphdr)) {
    // too short ICMP header
    printf("* ");
    fflush(stdout);
    continue;
}

// pointer to the ICMP header (right after the IP header)
struct icmphdr *ricmp = (struct icmphdr *)(recvbuf + rip_hlen);

// Convert source IP to string
char hop_ip[INET_ADDRSTRLEN];
inet_ntop(AF_INET, &from.sin_addr, hop_ip, sizeof(hop_ip));

// Determine if this ICMP message corresponds to our probe.
int is_ours = 0;

// Check the ICMP message type we received
if (ricmp->type == ICMP_ECHOREPLY) {
    // Verify that this reply matches our process ID
    if (ntohs(ricmp->un.echo.id) == my_id) {
        is_ours = 1; // This reply belongs to our probe
        reached_destination = 1; // We reached the final destination
    }
}
```

אם התקבלה הודעת ICMP Time Exceeded, המשמעות היא שה-TTL של החבילה פג בנתב ביניים, ולכן מדובר ב-hop במסלול.

בכל מקרה, מתבצע אימות שהתשובה שייכת לבקשה שנשלחה, באמצעות בדיקת מזהה ה-ICMP (id). בהודעות שגיאה, האימות מתבצע מול חבילת ה-ICMP המקורית המוטמעת בתוך ההודעה. תשובות שאינן תואמות לבקשות שלנו נזרקות.

עבור תשובות תקינות מחושב זמן ה-RTT, ומודפסים כתובת ה-IP של ה-hop וזמן ההשהיה. אם לא מתקבלת תשובה עבור probe מסוים, מודפס "*" , בדומה לפלט רגיל.

הפלט עבור כל hop כולל את מספר ה-hop, כתובת הנתב (אם קיימת), ושלושה זמני RTT או "*" בהתאם לתשובות.

כאמור הריצה נעצרת כאשר מתקבלת תשובת Echo Reply מהיעד או כאשר מגיעים למספר ה-hops המקסימלי (הגדרנו 30). בסיום התוכנית נסגרת בצורה מסודרת ונסגר ה-socket.

```
} else if (ricmp->type == ICMP_TIME_EXCEEDED || ricmp->type == ICMP_DEST_UNREACH) {
    // Make sure we have embedded original IP header:
    // Routers send TIME_EXCEEDED (TTL expired) or DEST_UNREACH
    // These ICMP messages contain the "original packet" inside them

    // Point to the embedded (inner) IP header inside the ICMP payload
    unsigned char *inner = (unsigned char *) (recvbuf + rip_hlen + sizeof(struct icmphdr));

    // Length of the embedded data
    ssize_t inner_len = n - (rip_hlen + (ssize_t)sizeof(struct icmphdr));

    // Make sure we have at least an IP header inside
    if (inner_len >= (ssize_t)sizeof(struct iphdr)) {
        // Parse the outer IP header from the received packet
        struct iphdr *inner_ip = (struct iphdr *)inner;
        // IP header length is stored in 32-bit words, so multiply by 4
        int inner_hlen = inner_ip->ihl * 4;

        // Make sure the embedded packet also contains an ICMP header
        if (inner_len >= inner_hlen + (ssize_t)sizeof(struct icmphdr)) {
            // pointer to the ICMP header (right after the IP header)
            struct icmphdr *inner_icmp = (struct icmphdr *) (inner + inner_hlen);

            // Verify that the embedded packet is our ICMP Echo Request
            if (inner_ip->protocol == IPPROTO_ICMP &&
                ntohs(inner_icmp->un.echo.id) == my_id) {
                // This ICMP error refers to our probe
                is_ours = 1;
            }
        }
    }
}

if (!is_ours) {
    // If the ICMP message is not related to our probe, treat as timeout for this probe.
    printf("* ");
    fflush(stdout);
    continue;
}

// Print hop IP once (typical traceroute style), unless it changes
if (hop_ip_first[0] == '\0') {
    // First response for this hop
    strncpy(hop_ip_first, hop_ip, sizeof(hop_ip_first) - 1);
    hop_ip_first[sizeof(hop_ip_first) - 1] = '\0';
    printf("%s ", hop_ip_first);
} else if (strcmp(hop_ip_first, hop_ip) != 0) {
    // Rare, but possible: different routers responding for different probes
    printf("%s ", hop_ip);
}

// Print the round-trip time for this probe
printf("%.2f ms ", rtt);
fflush(stdout);
}

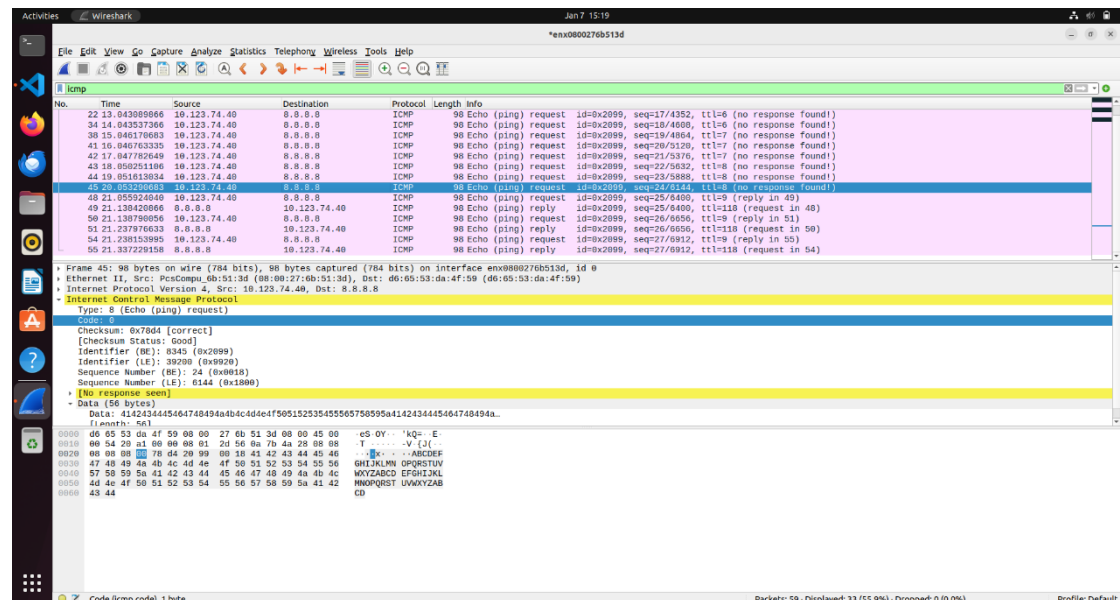
// End of probes for this TTL
printf("\n");

// Stop traceroute once the destination replied
if (reached_destination) {
    break;
}
}

// Close the raw socket before exiting
close(sock);
```

נריץ ונקבל כנדרש:

ניתן לראות כי לאחר ה-hop הראשון התוכנה מדפיסה מספר *, המעידים על נתבי ביניים שאינם מחזירים הודעות ICMP Time Exceeded תקינה כפי שהיינו מצפים, למרות שהחבילות ממשיכות לעבור דרכם. עם הגדלת ה-TTL ל-9 מתקבלת הודעת ICMP Echo Reply מהיעד 8.8.8.8, דבר המאשר את הגעת החבילה ליעד וסיום התוכנית. כעת ננתח את תעבורת הרשת:



ניתן לראות ב-Wireshark כי עבור חלק מערכי ה-TTL נשלחות חבילות ICMP Echo Request תקינות, אך לא מתקבלת תגובת ICMP מתאימה. תופעה זו נובעת מכך שנתבי ביניים מסוימים אינם מחזירים הודעות Time Exceeded, למרות שהחבילה דורקת או מועברת הלאה. בהתאם לכך, התוכנית מדפיסה * כפי שהסברנו.

חלק C – סורק פורטים

נמשיך לעבוד באותה סביבה של שני החלקים הקודמים במטלה. וכמובן נעזרו בקוד מהתרגולים לצורך חישוב checksum ושימוש במבני הנתונים של TCP/IP.

בנה תוכנה המבצעת סריקה של פורטים של TCP בטווח הפורטים 1 עד 65535, ומדפיסה למשתמש אילו פורטים פתוחים ביעד נתון. הסריקה מתבצעת על ידי שליחת חבילת TCP עם דגל SYN ובדיקת סוג התגובה המתקבלת מהיעד.

מפאת קוצר הזמן נתמוך רק ב-TCP בהגשה זו, אנו מצרים על כך.

התוכנית מקבלת פרמטר אחד מהמשתמש: פלאג a- ולאחריו כתובת יעד, שיכולה להיות כתובת IP או hostname. בשלב הראשון מתבצעת בדיקת תקינות של הפרמטרים, ובמידה והפרמטר חסר או לא תקין: התוכנית מדפיסה הודעת שגיאה מתאימה ומסתיימת. כאמור אנו תומכים כאן רק ב-TCP ולכן אין צורך לרשום פלאג נוסף של t- עם הגדרה של TCP או UDP.

```

int main(int argc, char **argv) {
    const char *host = NULL;
    int opt;

    // Parse command line arguments
    while ((opt = getopt(argc, argv, "a:")) != -1) {
        if (opt == 'a') host = optarg;        // Target host
        else {
            fprintf(stderr, "Usage: sudo %s -a <host>\n", argv[0]);
            return 1;
        }
    }

    // Validate required arguments
    if (!host) {
        fprintf(stderr, "Usage: sudo %s -a <host>\n", argv[0]);
        return 1;
    }
}

```

לאחר קבלת הפרמטרים, מתבצעת המרה של כתובת היעד לכתובת IPv4 בינארית. תחילה מתבצע ניסיון לפרש את המחרוזת ככתובת IP ישירה באמצעות `inet_pton`. אם הניסיון נכשל, מתבצע DNS lookup באמצעות `getaddrinfo`, כאשר התוצאה מוגבלת לIPv4 בלבד. בסיום שלב זה מתקבלת כתובת היעד במבנה `struct in_addr` (במידה ולא הצלחנו לקבל כתובת IP תקינה אז התוכנה נסגרת):

```

// Resolve hostname or IP string to IPv4 address
int resolve_ipv4(const char *host, struct in_addr *out) {
    // Try to parse as direct IP address first
    if (inet_pton(AF_INET, host, out) == 1) {
        // We got a valid IP already, no DNS lookup is required
        return 0;
    }

    // Otherwise, perform DNS resolution
    struct addrinfo hints = {.ai_family = AF_INET}, *res = NULL; // Restricts results to IPv4

    // Perform DNS resolution for the given hostname
    if (getaddrinfo(host, NULL, &hints, &res) || !res) {
        // DNS resolution failed
        return -1;
    }

    // Extract the IPv4 address from the result
    *out = ((struct sockaddr_in *)res->ai_addr)->sin_addr;

    // Free memory allocated by getaddrinfo
    freeaddrinfo(res);

    return 0; // Success - we now have a valid IP
}

```

כעת בשביל לבנות חבילות IP וTCP בעצמנו, יש צורך לדעת מהי כתובת הIP המקומית שממנה המחשב ישלח את החבילות אל היעד. לצורך כך השתמשנו בטריק שבו נפתח `UDP socket`, מבוצע `connect` ליעד (בפורט דמה), ולאחר מכן באמצעות `getsockname` נשלפת כתובת הIP המקומית שנבחרה על ידי מערכת ההפעלה למסלול זה. פעולה זו אינה שולחת חבילות בפועל, אך מאפשרת קבלת כתובת המקור הנכונה:

```

// Determines the local IPv4 address that would be used to send packets to a given destination
// Uses the "idle UDP socket trick" - no actual packets are sent
int get_local_ip(struct in_addr dst, struct in_addr *src) {
    // Create an IPv4 UDP socket (no packets are actually sent)
    int s = socket(AF_INET, SOCK_DGRAM, 0);
    if (s < 0) return -1;

    // Prepare destination address (IPv4, port 53 DNS as a dummy port)
    struct sockaddr_in d = {
        .sin_family = AF_INET,
        .sin_addr   = dst,
        .sin_port   = htons(53) // DNS port (arbitrary choice)
    };

    // "Connect" the UDP socket to force route selection
    // This doesn't send any data, just sets up routing
    if (connect(s, (struct sockaddr *)&d, sizeof(d)) < 0) {
        close(s);
        return -1;
    }

    // Length parameter required by getsockname
    socklen_t len = sizeof(d);

    // Query the socket for its locally assigned address
    if (getsockname(s, (struct sockaddr *)&d, &len) < 0) {
        close(s);
        return -1;
    }

    // Extract the local IPv4 address
    *src = d.sin_addr;

    // Close the socket
    close(s);

    return 0; // Success
}

```

בשלב הבא נפתחים שני Raw Sockets: שליחה וקבלה. עבור סוקט השליחה, כמו בסעיפים הקודמים בשביל שמערכת ההפעלה לא תיצור בעצמה את ה-IP header נדליק את הדגל IP_HDRINCL, ואנחנו נבנה אותו בעצמנו.

הסוקט השני משמש לקבלת תשובות ה-TCP, כך שניתן לקלוט תשובות SYN+ACK או RST מהיעד.

כמובן שיש צורך גם כאן ב-SUDO בשביל להפעיל את שני הסוקטים.

לצורך חישוב ה-checksum של ה-header השונים לקחנו שוב את פונקציית checksum מהתרגולים כאמור:

- עבור IPv4, ה-checksum מחושב על כותרת ה-IP בלבד (כי נתבים בדרך מעדכנים שדות של ה-IP כמו ה-TTL ולכן אינם יכולים לחשב checksum על הנתונים).
- עבור TCP מחושב ה-checksum הכולל pseudo header (כתובות מקור ויעד, פרוטוקול ואורך), יחד עם כותרת ה-TCP וה-payload במידה וקיים.

```

// Compute IP header checksum (only header, no payload)
uint16_t ip_checksum(struct iphdr *ip) {
    // Clear the checksum field before calculation
    ip->check = 0;

    // Compute checksum over the IPv4 header (ihl is in 32-bit words)
    return checksum(ip, ip->ihl * 4);
}

// Compute TCP checksum using pseudo header
uint16_t tcp_checksum(const struct iphdr *ip,
                      const struct tcphdr *tcp,
                      size_t len) {
    // Build the TCP pseudo-header required for checksum calculation
    struct pseudo_tcp ph = {
        ip->saddr,      // Source IPv4 address
        ip->daddr,      // Destination IPv4 address
        0,              // Reserved field (must be zero)
        IPPROTO_TCP,    // Protocol identifier for TCP
        htons(len)      // TCP segment length (header + payload)
    };

    // Temporary buffer for pseudo-header + TCP segment
    unsigned char buf[sizeof(ph) + sizeof(struct tcphdr)];

    // Copy pseudo-header into the buffer
    memcpy(buf, &ph, sizeof(ph));

    // Copy TCP header and payload after the pseudo-header
    memcpy(buf + sizeof(ph), tcp, len);

    // Compute and return the checksum over the combined buffer
    return checksum(buf, sizeof(ph) + len);
}

```

סריקת פורט בודד מתבצעת בפונקציה ייעודית, הבונה חבילת TCP SYN בזיכרון. החבילה כוללת כותרת IP עם ערכים קבועים כגון TTL, IPv4, כתובות המקור והיעד, וכותרת TCP שבה מוגדר דגל SYN, מספר רצף אקראי, ופורט יעד בהתאם לפורט הנסרק. לאחר חישוב checksums, החבילה נשלחת ליעד באמצעות sendto.

```

// Create raw socket for sending (with IP_HDRINCL to build our own headers)
int ss = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);
if (ss < 0) {
    perror("socket");
    return 1;
}

// Tell kernel we're providing our own IP headers
int one = 1;
if (setsockopt(ss, IPPROTO_IP, IP_HDRINCL, &one, sizeof(one)) < 0) {
    perror("setsockopt");
    close(ss);
    return 1;
}

// Create raw socket for receiving TCP responses
int rs = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);
if (rs < 0) {
    perror("socket");
    close(ss);
    return 1;
}

// Use one source port for the whole scan (simplifies matching)
uint16_t sp = 40000 + (getpid() % 20000);

// Scan all ports from 1 to 65535
for (int p = PORT_MIN; p <= PORT_MAX; p++) {
    // Scan the port
    if (scan_tcp(ss, rs, src, dst, sp, p)) {
        // Print if port is open
        printf("OPEN TCP %d\n", p);
        fflush(stdout); // Immediate output
    }

    // Progress indicator every 2000 ports
    if (p % 2000 == 0)
        fprintf(stderr, "Scanned %d/65535...\n", p);
}

// Clean up sockets
close(rs);
close(ss);

printf("Scan complete.\n");
return 0;

```

לאחר שליחת חבילת ה-SYN, התוכנית ממתינה לתשובה באמצעות poll עם timeout (קצר יחסית) של 30 מילישניות. במידה ומתקבלת חבילה, מתבצעת בדיקה שהיא אכן תשובה לפורט ולחבילה שנשלחו: כתובת המקור, כתובת היעד, פורט המקור ופורט היעד נבדקים כדי לוודא התאמה מלאה. רק חבילות העונות על תנאים אלו נבדקות.

אם מתקבלת תשובת TCP עם הדגלים SYN וACK, המשמעות היא שהפורט פתוח. במקרה זה התוכנית מסמנת את הפורט כפתוח ומדפיסה אותו למסך. אם מתקבלת תשובת RST, הפורט נחשב סגור. במידה ולא מתקבלת כל תשובה בתוך הזמן המוגדר, הפורט נחשב סגור מבינתנו.

לאחר מכן נשלח חבילת TCP מינימלית עם דגל RST בכל מקרה – בין אם הפורט פתוח, סגור או לא התקבלה תשובה (או timeout).

אבל במקרה של פורט פתוח (SYN+ACK), נשלחת חבילת RST עם ACK מתאים כדי "לסגור" את החיבור באופן תקין ולמנוע השארת החיבור פתוח בצד היעד.

```
int scan_tcp(int ss, int rs, struct in_addr src, struct in_addr dst, uint16_t sp, uint16_t p) {
    // Allocate packet buffer: IP header + TCP header (no payload)
    unsigned char pkt[sizeof(struct iphdr) + sizeof(struct tcphdr)] = {0};
    struct iphdr *ip = (struct iphdr *)pkt;
    struct tcphdr *tcp = (struct tcphdr *) (pkt + sizeof(struct iphdr));
    uint32_t seq = rand(); // Random initial sequence number

    // Build IP header
    ip->ihl = 5; // Header length (5 * 4 = 20 bytes)
    ip->version = 4; // IPv4
    ip->ttl = 64; // Time to live
    ip->protocol = IPPROTO_TCP; // TCP protocol
    ip->tot_len = htons(sizeof(pkt)); // Total packet length
    ip->id = htons(rand() & 0xFFFF); // Random IP ID
    ip->saddr = src.s_addr; // Source IP (our IP)
    ip->daddr = dst.s_addr; // Destination IP (target)
    ip->check = ip_checksum(ip); // Compute IP checksum

    // Build TCP SYN packet
    tcp->source = htons(sp); // Our source port
    tcp->dest = htons(p); // Target port
    tcp->seq = htonl(seq); // Our sequence number
    tcp->doff = 5; // Data offset (20 bytes)
    tcp->syn = 1; // SYN flag (connection request)
    tcp->window = htons(65535); // Maximum window size
    tcp->check = tcp_checksum(ip, tcp, sizeof(struct tcphdr));

    // Send the SYN packet
    struct sockaddr_in d = {.sin_family = AF_INET, .sin_addr = dst};
    if (sendto(ss, pkt, sizeof(pkt), 0, (struct sockaddr *)&d, sizeof(d)) < 0) return 0;

    // Wait for response using poll (non-blocking with timeout)
    struct pollfd pfd = {.fd = rs, .events = POLLIN};
    unsigned char buf[2048];
    int open = 0;

    // Poll for TCP_TIMEOUT_MS milliseconds
    if (poll(&pfd, 1, TCP_TIMEOUT_MS) > 0 && (pfd.revents & POLLIN)) {
        // Data is available to read
        ssize_t n = recvfrom(rs, buf, sizeof(buf), 0, NULL, NULL);

        if (n >= (ssize_t)sizeof(struct iphdr)) {
            struct iphdr *rip = (struct iphdr *)buf;
            int ihl = rip->ihl * 4; // IP header length in bytes

            // Make sure we have enough data for TCP header
            if (n >= ihl + (ssize_t)sizeof(struct tcphdr) && rip->protocol == IPPROTO_TCP) {
                struct tcphdr *rtcp = (struct tcphdr *) (buf + ihl);

                // Verify this response is for our probe
                if (rip->saddr == dst.s_addr && // From our target
                    rip->daddr == src.s_addr && // To us
                    ntohs(rtcp->source) == p && // From the port we scanned
                    ntohs(rtcp->dest) == sp) { // To our source port

                    uint32_t pseq = ntohs(rtcp->seq); // Their sequence number
                    uint32_t pack = ntohs(rtcp->ack_seq); // Their ack number

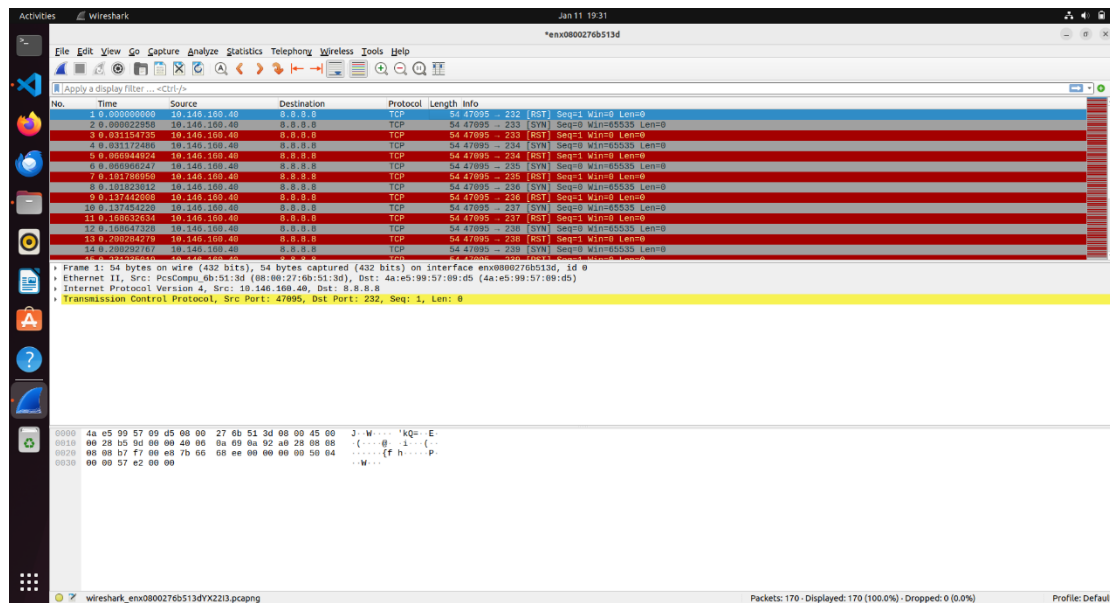
                    // SYN+ACK means port is OPEN
                    if (rtcp->syn && rtcp->ack) {
                        open = 1;
                        // Send RST+ACK to close the connection politely
                        // seq = what they acked, ack = their seq+1
                        send_rst(ss, src, dst, sp, p, pack, pseq + 1, 1);
                    }
                    // RST means port is CLOSED
                    else if (rtcp->rst) {
                        // Still send RST (assignment requirement: send RST either way)
                        send_rst(ss, src, dst, sp, p, pack, pseq, 1);
                    }
                }
            }
        }
    }
}
```


נפעיל את כל המנגנון הזה בלולאה הסורקת את כל טווח הפורטים מ1 עד 65535. לצורך הסריקה נבחר פורט מקור קבוע בטווח גבוה, כדי להקל על התאמת תשובות ולמנוע התנגשות עם שירותים ותוכנות אחרות שרצות במחשב. עבור כל פורט מתבצעת סריקה, ובמידה והפורט פתוח מודפסת הודעה מתאימה. בנוסף, התוכנית מדפיסה הודעת התקדמות כל מספר אלפי פורטים.

לאחר מכן, בסיום הסריקה נסגרים הסוקטים והתוכנית מסתיימת בצורה מסודרת.

נבחן את תעבורת הרשת בזמן ההרצה:

```
vboxuser@Ubuntu-22:~/Desktop/networks_4$ sudo ./port_scanning -a 8.8.8.8
Scanning 8.8.8.8 (8.8.8.8) with TCP...
OPEN TCP 53
OPEN TCP 80
```



ניתן לראות רצף חבילות TCP היוצאות מהמחשב שלנו ליעד, כאשר רוב החבילות מסומנות בצבע אדום עקב פלאג של RST. צבע זה מציין חבילות שמטרתן סגירת חיבור או ניקוי חצי-חיבור, בהתאם להתנהגות של סריקת ה-SYN שלנו. חבילות בודדות בצבע כחול מייצגות חבילות TCP רגילות כגון SYN.